

EXPLICACIÓN COMPLETA Y DETALLADA DEL CÓDIGO

Simulador de Gestión de Memoria Particionada Fija

1. VISIÓN GENERAL

¿Qué hace este programa?

Este programa simula cómo un **Sistema Operativo** gestiona la memoria RAM cuando varios programas (procesos) necesitan ejecutarse al mismo tiempo.

Analogía simple: Imagina que la memoria RAM es un edificio de departamentos. Cada departamento tiene el mismo tamaño (partición fija). Cuando llega un nuevo inquilino (proceso), el administrador (sistema operativo) le asigna el primer departamento disponible que encuentre.

Concepto Clave: Particionamiento Fijo

Particionamiento Fijo significa que:

- La memoria se divide en **bloques de tamaño igual** al inicio
- Estos bloques **NO cambian de tamaño** durante la ejecución
- Un proceso ocupa **UN bloque completo** (aunque no lo use todo)

Algoritmo Usado: First Fit (Primer Ajuste)

First Fit significa:

- Buscar la **primera** partición libre que sea suficientemente grande
- **No** buscar la mejor, solo la primera que sirva
- Es rápido pero puede desperdiciar espacio

2. ESTRUCTURAS DE DATOS

2.1 Estructura **Particion** (Líneas 7-13)

```
typedef struct {  
    int id;                /* Número único de la partición (0, 1, 2...) */  
    int id_proceso;        /* ¿Qué proceso la está usando? (-1 = nadie) */  
    int direccion_inicio;  /* ¿En qué posición de memoria comienza? */  
    int tamano;            /* ¿Cuántos KB tiene? */  
    int estado;            /* ¿Está libre (0) u ocupada (1)? */  
} Particion;
```

Explicación Simple: Es como la **ficha de un departamento** en un edificio:

- **id:** Número del departamento (Depto 1, Depto 2...)
- **id_proceso:** ¿Quién vive ahí? (Si es -1, está vacío)
- **direccion_inicio:** Dirección del departamento
- **tamano:** Tamaño del departamento (siempre igual en todos)
- **estado:** ¿Está ocupado (1) o libre (0)?

Ejemplo:

```

Particion depto_1 = {
    id = 0,
    id_proceso = 5,          // Vive el proceso 5
    direccion_inicio = 0,
    tamano = 100,           // 100 KB
    estado = 1              // Ocupada
};

```

2.2 Estructura Proceso (Líneas 15-20)

```

typedef struct Proceso {
    int id;                  /* Número único del proceso */
    int tamano_requerido;    /* ¿Cuánta memoria necesita? */
    int direccion_asignada;  /* ¿Dónde quedó en memoria? */
    struct Proceso *siguiente; /* Puntero al siguiente proceso */
} Proceso;

```

Explicación Simple: Es como la **ficha de un inquilino**:

- **id:** Identificación del inquilino (Persona 1, Persona 2...)
- **tamano_requerido:** ¿Cuánto espacio necesita realmente?
- **direccion_asignada:** ¿En qué dirección vive?
- **siguiente:** Puntero que conecta con el siguiente inquilino (lista enlazada)

¿Por qué es una lista enlazada? Porque NO sabemos cuántos procesos habrá. La lista **crece dinámicamente** según sea necesario.

Analogía: Es como una cadena de personas donde cada una tiene la mano sobre el hombro de la siguiente:

```

Proceso1 → Proceso2 → Proceso3 → NULL
    ↓           ↓           ↓
  ID:1        ID:2        ID:3
Tamaño:75   Tamaño:60   Tamaño:90

```

3. VARIABLES GLOBALES

3.1 Variables de Memoria (Líneas 24-31)

```

int *memoria = NULL;          /* Vector que simula la RAM */
Particion *tabla_particiones = NULL; /* Arreglo de particiones */
Proceso *lista_procesos = NULL; /* Lista de procesos activos */
Proceso *lista_procesos_terminados = NULL; /* Lista de procesos cerrados */
int num_particiones = 0;      /* ¿Cuántas particiones hay? */
int tamano_total_memoria = 0; /* Tamaño total de RAM */
int tamano_particion = 0;     /* Tamaño de cada partición */
int contador_procesos = 1;    /* Para generar IDs únicos */

```

Explicación Detallada:

int *memoria

Es un **vector (arreglo)** que simula la memoria RAM byte por byte.

Ejemplo: Si tienes 500 KB de memoria, tendrás un vector de 500 posiciones:

```
memoria[0] = -1    (libre)
memoria[1] = -1    (libre)
memoria[2] = 5     (ocupado por proceso 5)
memoria[3] = 5     (ocupado por proceso 5)
...
```

¿Por qué -1 significa libre? Porque los IDs de procesos empiezan en 1, entonces -1 es un valor que nunca usará un proceso real.

Particion *tabla_particiones

Es un **arreglo** de particiones. Cada posición contiene la información de UNA partición.

Ejemplo: Si hay 10 particiones:

```
tabla_particiones[0] = {id:0, estado:1, id_proceso:5, ...} (ocupada)
tabla_particiones[1] = {id:1, estado:0, id_proceso:-1, ...} (libre)
tabla_particiones[2] = {id:2, estado:1, id_proceso:3, ...} (ocupada)
...
```

Proceso *lista_procesos

Es una **lista enlazada** de los procesos que están **activos** (ejecutándose).

¿Por qué lista y no arreglo? Porque NO sabemos cuántos procesos habrá. Con una lista, podemos agregar y quitar procesos fácilmente.

Visualización:

lista_procesos → [P1] → [P2] → [P3] → NULL

P1: ID=1, Tamaño=75, Dirección=0
P2: ID=2, Tamaño=60, Dirección=100
P3: ID=3, Tamaño=90, Dirección=200

Proceso *lista_procesos_terminados

Igual que lista_procesos pero para procesos que YA terminaron (historial).

Utilidad: Permite ver qué procesos se ejecutaron en el pasado (opción 6 del menú).

int contador_procesos

Es un contador que genera IDs únicos automáticamente.

Ejemplo:

```
Crear proceso 1 → contador_procesos = 1
Crear proceso 2 → contador_procesos = 2
Crear proceso 3 → contador_procesos = 3
...
```

Importante: El contador NUNCA se reinicia, incluso si cierras procesos. Los IDs son únicos para siempre.

4. FUNCIÓN PRINCIPAL (main)

Líneas 51-103

```
int main(void) {
    int opcion;

    srand((unsigned int)time(NULL)); /* ← IMPORTANTE */

    printf("SIMULADOR DE GESTION DE MEMORIA...\n");

    inicializar_memoria(); /* ← Paso 1: Configurar memoria */

    do {
        /* Mostrar menú */
        printf("\n\n===== MENU PRINCIPAL =====\n");
        printf("  1. Crear Proceso\n");
        printf("  2. Cerrar Proceso\n");
        /* ... más opciones ... */

        scanf("%d", &opcion);

        switch(opcion) {
            case 1: crear_proceso(); break;
            case 2: cerrar_proceso(); break;
            /* ... más casos ... */
        }

    } while(opcion != 0);

    liberar_recursos(); /* ← Paso final: Limpiar memoria */

    return 0;
}
```

Explicación Paso a Paso:

1. `srand((unsigned int)time(NULL))` (Línea 54)

¿Qué hace? Inicializa el generador de números aleatorios.

¿Por qué? Porque cuando creamos un proceso, su tamaño se genera **aleatoriamente**. Si no ponemos `srand()`, siempre generará los mismos números.

Analogía: Es como "mezclar las cartas" antes de jugar. `time(NULL)` usa el reloj de la computadora como "semilla" para que cada vez sea diferente.

2. `inicializar_memoria()` (Línea 58)

¿Qué hace? Configura TODO el sistema de memoria ANTES de que el usuario pueda crear procesos.

¿Qué se configura?

- Pregunta al usuario el tamaño total de memoria
- Pregunta el tamaño de cada partición
- Crea el vector `memoria`
- Crea la tabla de particiones
- Marca todo como libre

3. Bucle `do-while` (Líneas 60-98)

¿Qué hace? Es el **menú principal** del programa. Se ejecuta **hasta que el usuario elija 0 (Salir)**.

Flujo:

1. Mostrar menú
2. Leer opción del usuario
3. Ejecutar la función correspondiente (switch)
4. Volver al paso 1 (si no eligió 0)

4. `switch(opcion)` (Líneas 72-96)

¿Qué hace? Ejecuta la función correspondiente según la opción elegida.

Desglose:

```
case 1: crear_proceso();           /* Crea un nuevo proceso */
case 2: cerrar_proceso();          /* Cierra un proceso existente */
case 3: mostrar_tabla_procesos();  /* Ver procesos activos */
case 4: mostrar_tabla_particiones(); /* Ver estado de particiones */
case 5: mostrar_memoria();         /* Ver memoria visualmente */
case 6: mostrar_procesos_terminados(); /* Ver historial */
case 0: /* Salir */                /* Termina el programa */
default: /* Opción inválida */     /* Mensaje de error */
```

5. `liberar_recursos()` (Línea 100)

¿Qué hace? Limpia toda la memoria que usó el programa ANTES de terminar.

¿Por qué es importante? Porque si no liberas la memoria, se produce una **fuga de memoria** (memory leak). Es como dejar las luces prendidas cuando sales de tu casa.

5. INICIALIZACIÓN DE MEMORIA

Función inicializar_memoria() (Líneas 107-167)

Esta es una de las funciones **MÁS IMPORTANTES** del programa. Se ejecuta **UNA SOLA VEZ** al inicio.

Paso a Paso Detallado:

PASO 1: Pedir Tamaño Total (Líneas 112-118)

```
do {
    printf("Ingrese el tamaño total de memoria (KB): ");
    scanf("%d", &tamano_total_memoria);
    if (tamano_total_memoria <= 0) {
        printf("Error: El tamaño debe ser mayor a 0.\n");
    }
} while (tamano_total_memoria <= 0);
```

¿Qué hace? Pregunta al usuario cuánta memoria RAM hay en total.

Validación: El bucle do-while se repite SI el usuario ingresa 0 o un número negativo. Solo sale cuando ingresa un número positivo.

Ejemplo:

```
Usuario ingresa: -5    → Error, pide nuevamente
Usuario ingresa: 0     → Error, pide nuevamente
Usuario ingresa: 1000  → Correcto, continúa
```

PASO 2: Pedir Tamaño de Partición (Líneas 120-129)

```
do {
    printf("Ingrese el tamaño de cada particion (KB): ");
    scanf("%d", &tamano_particion);
    if (tamano_particion <= 0) {
        printf("Error: El tamaño debe ser mayor a 0.\n");
    } else if (tamano_particion > tamano_total_memoria) {
        printf("Error: El tamaño de particion no puede ser mayor...\n");
        tamano_particion = 0; /* ← Forzar repetición */
    }
} while (tamano_particion <= 0);
```

¿Qué hace? Pregunta cuánto medirá cada partición (departamento).

Validaciones:

1. Debe ser mayor que 0
2. NO puede ser mayor que la memoria total

Ejemplo:

Memoria total: 1000 KB

```
Usuario ingresa: 0      → Error (debe ser > 0)
Usuario ingresa: 2000   → Error (mayor que 1000)
Usuario ingresa: 100    → Correcto
```

PASO 3: Calcular Número de Particiones (Línea 131)

```
num_particiones = tamano_total_memoria / tamano_particion;
```

¿Qué hace? Calcula cuántas particiones caben en la memoria total.

Ejemplo:

Memoria total: 1000 KB

Tamaño partición: 100 KB

Cálculo: $1000 / 100 = 10$ particiones

Nota: La división es **entera**, no decimal. Si sobra espacio, se ignora.

Ejemplo con sobra:

Memoria total: 1000 KB

Tamaño partición: 300 KB

Cálculo: $1000 / 300 = 3$ particiones (sobran 100 KB que no se usan)

PASO 4: Crear Vector de Memoria (Líneas 141-149)

```
memoria = (int *)malloc(tamano_total_memoria * sizeof(int));
if (memoria == NULL) {
    printf("Error: No se pudo asignar memoria.\n");
    exit(1);
}

for (i = 0; i < tamano_total_memoria; i++) {
    memoria[i] = -1; /* Marca todo como libre */
}
```

¿Qué hace?

1. **malloc()**: Pide memoria dinámica al sistema operativo
2. **Verifica**: Si no hay memoria disponible, el programa termina con error
3. **Inicializa**: Pone -1 en todas las posiciones (todo libre)

Ejemplo Visual:

Memoria total: 500 KB

```
memoria = [-1, -1, -1, -1, -1, ..., -1] (500 veces)
           ↑   ↑   ↑   ↑   ↑           ↑
           pos pos pos pos pos         pos
           0   1   2   3   4         499
```

PASO 5: Crear Tabla de Particiones (Líneas 151-164)

```

tabla_particiones = (Particion *)malloc(num_particiones * sizeof(Particion));
if (tabla_particiones == NULL) {
    printf("Error: No se pudo asignar memoria...\n");
    free(memoria); /* ← Limpia lo que ya creó */
    exit(1);
}

for (i = 0; i < num_particiones; i++) {
    tabla_particiones[i].id = i;
    tabla_particiones[i].id_proceso = -1; /* Libre */
    tabla_particiones[i].direccion_inicio = i * tamano_particion;
    tabla_particiones[i].tamano = tamano_particion;
    tabla_particiones[i].estado = 0; /* Libre */
}

```

¿Qué hace?

1. **Crea el arreglo** de particiones
2. **Inicializa cada partición** con sus valores

Ejemplo Completo:

Memoria total: 300 KB
 Tamaño partición: 100 KB
 Número de particiones: 3

Tabla de Particiones:

ID	ID Proceso	Dir. Inicio	Tamaño	Estado	
0	-1	0	100	0	← Libre
1	-1	100	100	0	← Libre
2	-1	200	100	0	← Libre

Cálculo de direccion_inicio:

Partición 0: $0 * 100 = 0$
 Partición 1: $1 * 100 = 100$
 Partición 2: $2 * 100 = 200$

6. GESTIÓN DE LISTAS ENLAZADAS

6.1 Agregar Proceso (Líneas 171-195)

```

void agregar_proceso(int id, int tamano, int direccion) {
    Proceso *nuevo;
    Proceso *actual;

    /* 1. Crear nuevo nodo */
    nuevo = (Proceso *)malloc(sizeof(Proceso));
    if (nuevo == NULL) {
        printf("Error: No se pudo asignar memoria...\n");
        return;
    }
}

```



```

/* 2. Llenar datos */
nuevo->id = id;
nuevo->tamano_requerido = tamano;
nuevo->direccion_asignada = direccion;
nuevo->siguiente = NULL;

/* 3. Agregar a la lista */
if (lista_procesos == NULL) {
    lista_procesos = nuevo; /* Primera vez, es el único */
} else {
    /* Ir al final de la lista */
    actual = lista_procesos;
    while (actual->siguiente != NULL) {
        actual = actual->siguiente;
    }
    actual->siguiente = nuevo; /* Agregar al final */
}
}

```

Explicación Simple:

Imagina que es como **agregar un vagón a un tren**:

Caso 1: Lista vacía (primer proceso)

ANTES: NULL
 DESPUÉS: [P1] → NULL

Caso 2: Lista con procesos (agregar al final)

ANTES: [P1] → [P2] → NULL
 ↑
 Ir al final

DESPUÉS: [P1] → [P2] → [P3] → NULL
 ↑
 Nuevo agregado

¿Por qué al final? Para mantener el **orden cronológico**: el primer proceso creado queda primero, el segundo queda segundo, etc.

6.2 Buscar Proceso (Líneas 229-240)

```

Proceso* buscar_proceso(int id) {
    Proceso *actual;

    actual = lista_procesos;
    while (actual != NULL) {
        if (actual->id == id) {
            return actual; /* ← Encontrado */
        }
        actual = actual->siguiente;
    }
}

```

```

    return NULL; /* ← No encontrado */
}

```

Explicación Simple:

Es como **buscar un vagón específico en un tren**:

Buscar ID = 2

```

[P1] → [P2] → [P3] → NULL
  ↓      ↓      ↓
ID=1    ID=2    ID=3
      ↑
    Encontrado!

```

Proceso:

1. Empezar en el primer nodo
 2. ¿Es el que busco? → Sí: retornar / NO: continuar
 3. Ir al siguiente nodo
 4. Repetir hasta encontrar o llegar al final (NULL)
-

6.3 Eliminar Proceso (Líneas 197-227)

```

int eliminar_proceso(int id) {
    Proceso *actual;
    Proceso *anterior;
    Proceso *temp;

    if (lista_procesos == NULL) {
        return 0; /* Lista vacía */
    }

    actual = lista_procesos;
    anterior = NULL;

    /* Buscar el proceso */
    while (actual != NULL && actual->id != id) {
        anterior = actual;
        actual = actual->siguiente;
    }

    if (actual == NULL) {
        return 0; /* No encontrado */
    }

    /* Eliminar el nodo */
    if (anterior == NULL) {
        /* Es el primero */
        lista_procesos = actual->siguiente;
    } else {
        /* Es intermedio o final */
        anterior->siguiente = actual->siguiente;
    }
}

```

```

    free(actual); /* Liberar memoria */
    return 1;     /* Éxito */
}

```

Explicación Visual:

Caso 1: Eliminar el primero

ANTES: [P1] → [P2] → [P3] → NULL
 ↓
 Eliminar

DESPUÉS: [P2] → [P3] → NULL
 ↑
 lista_procesos apunta aquí ahora

Caso 2: Eliminar uno intermedio

ANTES: [P1] → [P2] → [P3] → NULL
 ↓
 Eliminar

DESPUÉS: [P1] -----> [P3] → NULL
 ↑ ↑
 anterior siguiente

Se "salta" el nodo eliminado

¿Por qué `free(actual)`? Porque al eliminar el nodo, debemos **liberar la memoria** que ocupaba. Si no, queda memoria "huérfana" (memory leak).

7. CREAR PROCESO (Asignación)

Función `crear_proceso()` (Líneas 283-329)

Esta es la función **MÁS COMPLEJA** e importante. Aquí se implementa el algoritmo **First Fit**.

Paso a Paso SUPER Detallado:

PASO 1: Generar Tamaño Aleatorio (Línea 290)

```
tamano_proceso = (rand() % (tamano_particion+0)) + 1;
```

¿Qué hace? Genera un número aleatorio entre 1 y el tamaño de la partición.

Explicación de la fórmula:

```
rand() % (tamano_particion + 0)
```

- `rand()`: Genera un número aleatorio grande (ej: 12345)
- `% tamano_particion`: Módulo, da el residuo de la división
- `+ 1`: Suma 1 para que nunca sea 0

Ejemplo:

```
tamano_particion = 100

rand() = 12345
12345 % 100 = 45
45 + 1 = 46 KB ← Tamaño del proceso
```

Rango: El tamaño estará entre 1 y 100 KB (nunca 0, nunca más de 100).

PASO 2: Buscar Partición Libre (Líneas 296-303)

```
particion_asignada = -1; /* -1 = no encontrada */

for (i = 0; i < num_particiones; i++) {
    if (tabla_particiones[i].estado == 0 && /* ← Está libre */
        tabla_particiones[i].tamano >= tamano_proceso) { /* ← Cabe */
        particion_asignada = i;
        break; /* ← IMPORTANTE: sale al encontrar la primera */
    }
}
```

¿Qué hace? Recorre las particiones **desde el inicio** hasta encontrar la **primera** que esté **libre** Y sea **suficientemente grande**.

Explicación del break: Este break es lo que hace que sea **First Fit**. Apenas encuentra una partición que sirve, **SALE** del bucle y NO sigue buscando.

Ejemplo Visual:

Proceso necesita: 60 KB
Tamaño de particiones: 100 KB cada una

Tabla de Particiones:

ID	Estado	Tamaño	
0	1	100	← Ocupada, salta
1	1	100	← Ocupada, salta
2	0	100	← Libre y >= 60, ASIGNA AQUÍ ✓
3	0	100	← No llega aquí (break)
4	0	100	← No llega aquí (break)

Resultado: particion_asignada = 2

PASO 3: Verificar si Encontró (Líneas 305-309)

```
if (particion_asignada == -1) {
    printf("ERROR: No hay particiones disponibles...\n");
    printf("    El proceso no pudo ser creado.\n");
    return; /* ← Sale de la función sin crear nada */
}
```

}

¿Qué hace? Si NO encontró ninguna partición libre, muestra error y **termina** la función.

Casos donde NO encuentra:

1. Todas las particiones están ocupadas
2. Las particiones libres son muy pequeñas para el proceso

PASO 4: Marcar Partición como Ocupada (Líneas 311-312)

```
tabla_particiones[particion_asignada].estado = 1;          /* Ocupada */
tabla_particiones[particion_asignada].id_proceso = contador_procesos;
```

¿Qué hace? "Reserva" la partición para este proceso.

Ejemplo:

ANTES:

ID	Estado	ID Proceso
2	0	-1

 ← Libre

DESPUÉS:

ID	Estado	ID Proceso
2	1	5

 ← Ocupada por proceso 5

PASO 5: Marcar en Memoria (Líneas 314-317)

```
dir_inicio = tabla_particiones[particion_asignada].direccion_inicio;

for (i = 0; i < tamano_proceso; i++) {
    memoria[dir_inicio + i] = contador_procesos;
}
```

¿Qué hace? Marca en el vector de memoria qué espacios están ocupados por este proceso.

Ejemplo Completo:

Partición asignada: 2
Dirección inicio: 200
Tamaño proceso: 60 KB
ID proceso: 5

ANTES (memoria[200...299] estaba libre):
memoria[200] = -1
memoria[201] = -1

```
...
memoria[259] = -1
memoria[260] = -1
...
memoria[299] = -1

DESPUÉS (memoria[200...259] ocupada por proceso 5):
memoria[200] = 5 ← Proceso 5
memoria[201] = 5 ← Proceso 5
...
memoria[259] = 5 ← Proceso 5
memoria[260] = -1 ← Libre (fragmentación interna)
...
memoria[299] = -1 ← Libre (fragmentación interna)
```

Nota: Solo marca tamaño_proceso posiciones, NO toda la partición. El resto queda como fragmentación interna.

PASO 6: Agregar a Lista (Línea 319)

```
agregar_proceso(contador_procesos, tamaño_proceso, dir_inicio);
```

¿Qué hace? Agrega el proceso a la lista enlazada de procesos activos.

PASO 7: Calcular Fragmentación (Línea 321)

```
fragmentacion = calcular_fragmentacion(tamaño_particion, tamaño_proceso);
```

¿Qué es fragmentación interna? Es el espacio **desperdiciado** dentro de una partición.

Fórmula:

$$\text{Fragmentación} = (\text{Tamaño_Partición} - \text{Tamaño_Proceso}) / \text{Tamaño_Partición} * 100\%$$

Ejemplo:

Tamaño partición: 100 KB

Tamaño proceso: 60 KB

Desperdicio: $100 - 60 = 40$ KB

Porcentaje: $(40 / 100) * 100 = 40\%$

Interpretación: Se desperdicia el 40% del espacio de la partición.

PASO 8: Mostrar Resultado (Líneas 323-327)

```
printf(">>> PROCESO CREADO EXITOSAMENTE <<<\n");
printf("    Asignado a la particion: %d\n", particion_asignada);
printf("    Direccion de inicio: %d\n", dir_inicio);
printf("    Fragmentacion interna: %.2f%%\n", fragmentacion);
```

PASO 9: Incrementar Contador (Línea 328)

```
contador_procesos++;
```

¿Por qué? Para que el **próximo** proceso tenga un ID diferente.

Ejemplo:

```
Proceso 1 creado → contador = 1 → al final: contador = 2
Proceso 2 creado → contador = 2 → al final: contador = 3
Proceso 3 creado → contador = 3 → al final: contador = 4
```

8. CERRAR PROCESO (Liberación)

Función `cerrar_proceso()` (Líneas 331-380)

Esta función **libera** una partición ocupada y la marca como disponible nuevamente.

Paso a Paso Detallado:

PASO 1: Pedir ID (Líneas 339-341)

```
printf("\n--- CERRAR PROCESO ---\n");
printf("Ingrese el ID del proceso a cerrar: ");
scanf("%d", &id_proceso);
```

PASO 2: Buscar Proceso (Líneas 343-347)

```
proceso = buscar_proceso(id_proceso);
if (proceso == NULL) {
    printf("ERROR: No existe un proceso con ID %d.\n", id_proceso);
    return;
}
```

¿Qué hace? Busca el proceso en la lista enlazada. Si no existe, muestra error y termina.

PASO 3: Obtener Información (Líneas 349-350)

```
tamano_proceso = proceso->tamano_requerido;
dir_inicio = proceso->direccion_asignada;
```

¿Por qué? Para saber qué espacio de memoria limpiar después.

PASO 4: Buscar Partición (Líneas 352-358)

```
particion_encontrada = -1;
```

```

for (i = 0; i < num_particiones; i++) {
    if (tabla_particiones[i].id_proceso == id_proceso) {
        particion_encontrada = i;
        break;
    }
}

```

¿Qué hace? Busca cuál partición contiene este proceso.

PASO 5: Liberar Partición (Líneas 365-366)

```

tabla_particiones[particion_encontrada].estado = 0;      /* Libre */
tabla_particiones[particion_encontrada].id_proceso = -1; /* Nadie */

```

Ejemplo:

ANTES:

ID	Estado	ID Proceso	
2	1	5	← Ocupada por P5

DESPUÉS:

ID	Estado	ID Proceso	
2	0	-1	← Libre

PASO 6: Limpiar Memoria (Líneas 368-372)

```

for (i = 0; i < tamaño_particion; i++) {
    if (memoria[dir_inicio + i] == id_proceso) {
        memoria[dir_inicio + i] = -1;
    }
}

```

¿Qué hace? Marca como libre (-1) todo el espacio que ocupaba el proceso.

Ejemplo:

Proceso 5 en dirección 200, tamaño 60 KB

ANTES:

```

memoria[200] = 5
memoria[201] = 5
...
memoria[259] = 5
memoria[260] = -1  (ya era libre, fragmentación)
...

```

DESPUÉS:


```
memoria[200] = -1  ← Libre
memoria[201] = -1  ← Libre
...
memoria[259] = -1  ← Libre
memoria[260] = -1  ← Libre
...
```

PASO 7: Historial (Línea 374)

```
agregar_proceso_terminado(id_proceso, tamano_proceso, dir_inicio);
```

¿Qué hace? Guarda el proceso en la lista de "terminados" para tener historial.

PASO 8: Eliminar de Lista Activa (Línea 376)

```
eliminar_proceso(id_proceso);
```

¿Qué hace? Quita el proceso de la lista de procesos activos.

9. VISUALIZACIÓN

9.1 Mostrar Tabla de Procesos (Líneas 384-404)

```
void mostrar_tabla_procesos(void) {
    Proceso *actual;

    printf("\n===== TABLA DE PROCESOS =====\n");
    printf("%-10s %-15s %-20s\n", "ID", "Tamaño (KB)", "Direccion Inicio");

    if (lista_procesos == NULL) {
        printf("No hay procesos activos.\n");
    } else {
        actual = lista_procesos;
        while (actual != NULL) {
            printf("%-10d %-15d %-20d\n",
                actual->id,
                actual->tamano_requerido,
                actual->direccion_asignada);
            actual = actual->siguiente;
        }
    }
}
```

¿Qué muestra? Una tabla con todos los procesos activos.

Ejemplo de Salida:

```
===== TABLA DE PROCESOS =====
ID          Tamaño (KB)      Direccion Inicio
-----
```

1	75	0
2	60	100
3	90	200

=====

9.2 Mostrar Tabla de Particiones (Líneas 406-446)

¿Qué muestra? Estado completo de cada partición, incluyendo fragmentación.

Ejemplo de Salida:

```
===== TABLA DE PARTICIONES =====
Part ID  Estado      Dir. Inicio (KB)  Tamaño (KB)  ID Proceso  Fragment. (%)
-----
0        Ocupada     0                100          1           25.00
1        Ocupada     100              100          2           40.00
2        Libre       200              100          ---         ---
3        Ocupada     300              100          3           10.00
=====
```

9.3 Mostrar Memoria (Líneas 448-468)

¿Qué muestra? Visualización del vector de memoria.

Ejemplo de Salida:

```
===== VISUALIZACION DE MEMORIA =====
Leyenda: [0] = Libre, [N] = Proceso ID N
-----
[1] [1] [1] [0] [0] [2] [2] [0] [0] [0]
[0] [0] [3] [3] [3] [3] [0] [0] [0] [0]
=====
```

Explicación:

- [1] = Ocupado por proceso 1
- [2] = Ocupado por proceso 2
- [0] = Libre

10. LIMPIEZA DE RECURSOS

Función liberar_recursos () (Líneas 498-524)

```
void liberar_recursos(void) {
    Proceso *actual;
    Proceso *temp;

    /* Liberar lista de procesos activos */
    actual = lista_procesos;
    while (actual != NULL) {
        temp = actual;
```

```

        actual = actual->siguiente;
        free(temp); /* ← Libera cada nodo */
    }

    /* Liberar lista de procesos terminados */
    actual = lista_procesos_terminados;
    while (actual != NULL) {
        temp = actual;
        actual = actual->siguiente;
        free(temp); /* ← Libera cada nodo */
    }

    /* Liberar vector de memoria */
    if (memoria != NULL) {
        free(memoria);
    }

    /* Liberar tabla de particiones */
    if (tabla_particiones != NULL) {
        free(tabla_particiones);
    }

    printf("\n>>> Recursos liberados. Programa finalizado.\n");
}

```

¿Por qué es importante? Porque toda la memoria que se pidió con `malloc()` debe devolverse con `free()`.

Analogía: Es como devolver los libros que pediste prestados en la biblioteca antes de irte a casa.

11. FLUJO COMPLETO CON EJEMPLO

Vamos a ver un ejemplo COMPLETO paso a paso:

Configuración Inicial:

Memoria total: 500 KB
 Tamaño partición: 100 KB
 Número de particiones: 5

Estado Inicial:

Tabla de Particiones:

ID	Estado	Tamaño	ID Proceso
0	0	100	-1
1	0	100	-1
2	0	100	-1
3	0	100	-1
4	0	100	-1

Memoria:
 [-1, -1, -1, ..., -1] (500 veces, todo libre)

Lista de procesos: NULL (vacía)

ACCIÓN 1: Crear Proceso P1 (75 KB)

Paso 1: Generar tamaño → 75 KB

Paso 2: Buscar partición → Encuentra partición 0 (primera libre)

Paso 3: Asignar partición 0 a P1

Paso 4: Marcar memoria[0...74] = 1

Paso 5: Agregar P1 a lista

Resultado:

Tabla de Particiones:

ID	Estado	ID Proceso
0	1	1
1	0	-1
2	0	-1
3	0	-1
4	0	-1

← P1

Memoria:

[1,1,1,...1,-1,-1,...,-1,-1,...,-1]
← P1 (75) → ←25 libre→ ←400 libre→

Fragmentación P1: $(100-75)/100 = 25\%$

Lista: [P1] → NULL

ACCIÓN 2: Crear Proceso P2 (60 KB)

Resultado:

Tabla de Particiones:

ID	Estado	ID Proceso
0	1	1
1	1	2
2	0	-1
3	0	-1
4	0	-1

← P1

← P2

Lista: [P1] → [P2] → NULL

ACCIÓN 3: Crear Proceso P3 (90 KB)

Resultado:

Tabla de Particiones:

ID	Estado	ID Proceso
0	1	1
1	1	2
2	1	3
3	0	-1
4	0	-1

← P1
← P2
← P3

Lista: [P1] → [P2] → [P3] → NULL

ACCIÓN 4: Cerrar Proceso P2

Paso 1: Buscar P2 en lista → Encontrado

Paso 2: Buscar partición de P2 → Partición 1

Paso 3: Liberar partición 1

Paso 4: Limpiar memoria[100...159] = -1

Paso 5: Agregar P2 a lista terminados

Paso 6: Eliminar P2 de lista activos

Resultado:

Tabla de Particiones:

ID	Estado	ID Proceso
0	1	1
1	0	-1
2	1	3
3	0	-1
4	0	-1

← P1
← LIBRE
← P3

Lista activos: [P1] → [P3] → NULL

Lista terminados: [P2] → NULL

ACCIÓN 5: Crear Proceso P4 (80 KB)

Paso 1: Buscar partición libre → Encuentra partición 1 (First Fit)

Paso 2: Asignar partición 1 a P4

Resultado:

Tabla de Particiones:

ID	Estado	ID Proceso
0	1	1
1	1	4
2	1	3
3	0	-1
4	0	-1

← P1
← P4 (reutilizó partición de P2)
← P3

Lista activos: [P1] → [P3] → [P4] → NULL

POSIBLES CAMBIOS DURANTE LA EVALUACIÓN

Guía Completa de Modificaciones Rápidas

ÍNDICE DE CAMBIOS

1. [Cambiar el Algoritmo de Asignación](#)
 2. [Modificar Tamaño de Procesos](#)
 3. [Cambiar Visualización de Memoria](#)
 4. [Agregar Estadísticas](#)
 5. [Modificar Validaciones](#)
 6. [Cambiar Fragmentación](#)
 7. [Agregar Prioridades](#)
 8. [Modificar Menú](#)
 9. [Cambiar Formato de Salida](#)
 10. [Agregar Límites](#)
-

1. CAMBIAR ALGORITMO DE ASIGNACIÓN

CAMBIO 1: De First Fit a Best Fit

¿Qué es Best Fit? En lugar de tomar la **primera** partición que sirva, busca la **más pequeña** que sirva (menor desperdicio).

Línea a modificar: 296-303 (función crear_proceso)

CÓDIGO ORIGINAL (First Fit):

```
particion_asignada = -1;
for (i = 0; i < num_particiones; i++) {
    if (tabla_particiones[i].estado == 0 &&
        tabla_particiones[i].tamano >= tamano_proceso) {
        particion_asignada = i;
        break; /* ← Primera que sirve */
    }
}
```

CÓDIGO NUEVO (Best Fit):

```
particion_asignada = -1;
int menor_desperdicio = tamano_total_memoria + 1; /* ← Agregar esta línea */
int desperdicio; /* ← Agregar esta línea */

for (i = 0; i < num_particiones; i++) {
    if (tabla_particiones[i].estado == 0 &&
        tabla_particiones[i].tamano >= tamano_proceso) {
```

```

        /* Calcular desperdicio */
        desperdicio = tabla_particiones[i].tamano - tamano_proceso;

        /* ¿Es mejor que la anterior? */
        if (desperdicio < menor_desperdicio) {
            menor_desperdicio = desperdicio;
            particion_asignada = i;
        }
        /* ← NO HAY BREAK, debe revisar todas */
    }
}

```

Explicación:

- Best Fit debe revisar **TODAS** las particiones
- Guarda la que tiene **menor desperdicio**
- Es más lento pero más eficiente en uso de memoria

Ejemplo:

Proceso necesita: 60 KB

Particiones disponibles:

- Part 1: 100 KB → Desperdicio: 40 KB
- Part 3: 80 KB → Desperdicio: 20 KB ← MEJOR (Best Fit)
- Part 5: 90 KB → Desperdicio: 30 KB

First Fit elegiría Part 1 (primera)

Best Fit elige Part 3 (menor desperdicio)

✚ CAMBIO 2: De First Fit a Worst Fit

¿Qué es Worst Fit? Busca la partición **MÁS GRANDE** disponible (peor ajuste, pero deja huecos grandes).

CÓDIGO NUEVO (Worst Fit):

```

particion_asignada = -1;
int mayor_tamano = -1; /* ← Agregar esta línea */

for (i = 0; i < num_particiones; i++) {
    if (tabla_particiones[i].estado == 0 &&
        tabla_particiones[i].tamano >= tamano_proceso) {

        /* ¿Es más grande que la anterior? */
        if (tabla_particiones[i].tamano > mayor_tamano) {
            mayor_tamano = tabla_particiones[i].tamano;
            particion_asignada = i;
        }
    }
}

```

Explicación:

- Busca la partición más grande

- Deja "huecos grandes" al desperdiciar
 - Evita fragmentos muy pequeños
-

✦ CAMBIO 3: Agregar Selector de Algoritmo

Agregar al inicio (después de línea 31):

```
int algoritmo = 1; /* 1=First, 2=Best, 3=Worst */
```

Modificar main () para preguntar (después de línea 54):

```
printf("Seleccione algoritmo:\n");
printf("1. First Fit (Primera que sirve)\n");
printf("2. Best Fit (Menor desperdicio)\n");
printf("3. Worst Fit (Particion mas grande)\n");
printf("Opcion: ");
scanf("%d", &algoritmo);
```

Modificar crear_proceso () con switch (línea 296):

```
switch(algoritmo) {
    case 1:
        /* Código First Fit aquí */
        break;
    case 2:
        /* Código Best Fit aquí */
        break;
    case 3:
        /* Código Worst Fit aquí */
        break;
}
```

2. MODIFICAR TAMAÑO DE PROCESOS

✦ CAMBIO 4: Tamaño Fijo en Lugar de Aleatorio

Línea a modificar: 290

CÓDIGO ORIGINAL:

```
tamano_proceso = (rand() % (tamano_particion+0)) + 1;
```

CÓDIGO NUEVO (Tamaño Fijo de 50 KB):

```
tamano_proceso = 50; /* Siempre 50 KB */
```

CÓDIGO NUEVO (Preguntar al Usuario):

```
printf("Ingrese el tamaño del proceso (KB): ");
scanf("%d", &tamano_proceso);

/* Validar */
```



```
if (tamano_proceso <= 0 || tamano_proceso > tamano_particion) {  
    printf("Error: Tamaño invalido.\n");  
    return;  
}
```

✦ CAMBIO 5: Rango Personalizado de Tamaño Aleatorio

CÓDIGO ORIGINAL:

```
tamano_proceso = (rand() % (tamano_particion+0)) + 1;
```

CÓDIGO NUEVO (Entre 50 y 100 KB):

```
tamano_proceso = (rand() % 51) + 50; /* 50 a 100 */
```

Explicación de la fórmula:

rand() % 51	→ Da 0 a 50
+ 50	→ Da 50 a 100

CÓDIGO NUEVO (Variable):

```
int min = 20;  
int max = 80;  
tamano_proceso = (rand() % (max - min + 1)) + min;
```

✦ CAMBIO 6: Tamaños Predefinidos (Array)

Agregar al inicio:

```
int tamanos_disponibles[] = {25, 50, 75, 100};  
int num_tamanos = 4;
```

Modificar línea 290:

```
int indice = rand() % num_tamanos;  
tamano_proceso = tamanos_disponibles[indice];
```

Explicación: El proceso tendrá uno de estos tamaños: 25, 50, 75 o 100 KB (aleatorio).

3. CAMBIAR VISUALIZACIÓN DE MEMORIA

✦ CAMBIO 7: Mostrar ID Real del Proceso

Línea a modificar: 463

CÓDIGO ORIGINAL:

```
printf("[1] ", memoria[i]); /* Siempre muestra [1] */
```

CÓDIGO NUEVO:

```
printf("[%d] ", memoria[i]); /* Muestra ID real: [5], [3], etc. */
```

Resultado:

```
ANTES: [1] [1] [1] [0] [0] [1] [1]
AHORA: [5] [5] [5] [0] [0] [3] [3]
        ↑         ↑
      Proceso 5   Proceso 3
```

✦ CAMBIO 8: Visualización con Colores (Avanzado)

Agregar al inicio del archivo:

```
#define COLOR_RESET    "\x1b[0m"
#define COLOR_RED      "\x1b[31m"
#define COLOR_GREEN    "\x1b[32m"
#define COLOR_YELLOW   "\x1b[33m"
```

Modificar línea 460-463:

```
if (memoria[i] == -1) {
    printf(COLOR_GREEN "[L] " COLOR_RESET); /* Verde para libre */
} else {
    printf(COLOR_RED "[%d] " COLOR_RESET, memoria[i]); /* Rojo para ocupado */
}
```

✦ CAMBIO 9: Visualización por Particiones

Modificar función `mostrar_memoria()`:

```
void mostrar_memoria(void) {
    int i, j;

    printf("\n===== VISUALIZACION POR PARTICIONES =====\n");

    for (i = 0; i < num_particiones; i++) {
        printf("Partición %d [", i);

        int inicio = tabla_particiones[i].direccion_inicio;

        for (j = 0; j < 10; j++) { /* Solo muestra primeros 10 */
            if (memoria[inicio + j] == -1) {
                printf("_");
            } else {
                printf("#");
            }
        }

        if (tabla_particiones[i].estado == 1) {
            printf("] Proceso %d\n", tabla_particiones[i].id_proceso);
        } else {

```

```

        printf("] Libre\n");
    }
}

printf("=====\n");
}

```

Resultado:

```

===== VISUALIZACION POR PARTICIONES =====
Particion 0 [#####] Proceso 1
Particion 1 [_____] Libre
Particion 2 [#####] Proceso 3
=====

```

4. AGREGAR ESTADÍSTICAS

✦ CAMBIO 10: Mostrar Estadísticas Generales

Agregar nueva función:

```

void mostrar_estadisticas(void) {
    int i;
    int particiones_ocupadas = 0;
    int particiones_libres = 0;
    int memoria_usada = 0;
    int memoria_libre = 0;
    float porcentaje_uso;
    Proceso *actual;
    int num_procesos_activos = 0;

    /* Contar particiones */
    for (i = 0; i < num_particiones; i++) {
        if (tabla_particiones[i].estado == 1) {
            particiones_ocupadas++;
        } else {
            particiones_libres++;
        }
    }

    /* Contar procesos activos */
    actual = lista_procesos;
    while (actual != NULL) {
        num_procesos_activos++;
        memoria_usada += actual->tamano_requerido;
        actual = actual->siguiente;
    }

    memoria_libre = tamano_total_memoria - memoria_usada;
    porcentaje_uso = (memoria_usada * 100.0) / tamano_total_memoria;

    printf("\n===== ESTADISTICAS DEL SISTEMA =====\n");
    printf("Memoria total: %d KB\n", tamano_total_memoria);
    printf("Memoria usada: %d KB (%.2f%%)\n", memoria_usada, porcentaje_uso);
    printf("Memoria libre: %d KB\n", memoria_libre);
}

```

```

printf("\n");
printf("Total particiones: %d\n", num_particiones);
printf("Particiones ocupadas: %d\n", particiones_ocupadas);
printf("Particiones libres: %d\n", particiones_libres);
printf("\n");
printf("Procesos activos: %d\n", num_procesos_activos);
printf("Procesos terminados: %d\n", contador_procesos - 1 -
num_procesos_activos);
printf("=====\n");
}

```

Agregar al menú (línea 67):

```
printf(" 7. Ver Estadísticas\n");
```

Agregar al switch (línea 89):

```

case 7:
    mostrar_estadisticas();
    break;

```

CAMBIO 11: Fragmentación Promedio

Agregar función:

```

void mostrar_fragmentacion_promedio(void) {
    int i;
    float frag_total = 0.0;
    int particiones_ocupadas = 0;
    Proceso *proc;

    for (i = 0; i < num_particiones; i++) {
        if (tabla_particiones[i].estado == 1) {
            proc = buscar_proceso(tabla_particiones[i].id_proceso);
            if (proc != NULL) {
                frag_total += calcular_fragmentacion(tabla_particiones[i].tamano,
                                                    proc->tamano_requerido);
                particiones_ocupadas++;
            }
        }
    }

    if (particiones_ocupadas > 0) {
        float frag_promedio = frag_total / particiones_ocupadas;
        printf("\nFragmentacion interna promedio: %.2f%%\n", frag_promedio);
    } else {
        printf("\nNo hay particiones ocupadas.\n");
    }
}

```

5. MODIFICAR VALIDACIONES

CAMBIO 12: Límite de Procesos Activos

Agregar al inicio:

```
int max_procesos = 5; /* Máximo 5 procesos simultáneos */
```

Modificar crear_proceso() (después de línea 283):

```
void crear_proceso(void) {
    int tamano_proceso;
    int particion_asignada;
    int i;
    int dir_inicio;
    float fragmentacion;
    Proceso *actual;
    int num_activos = 0;

    /* NUEVA VALIDACIÓN: Contar procesos activos */
    actual = lista_procesos;
    while (actual != NULL) {
        num_activos++;
        actual = actual->siguiente;
    }

    if (num_activos >= max_procesos) {
        printf("\nERROR: Maximo de %d procesos alcanzado.\n", max_procesos);
        printf("        Cierre un proceso antes de crear otro.\n");
        return;
    }

    /* ... resto del código ... */
}
```

✚ CAMBIO 13: Memoria Mínima Requerida

Agregar después de línea 290:

```
/* Validar memoria mínima */
int memoria_libre_total = 0;
for (i = 0; i < num_particiones; i++) {
    if (tabla_particiones[i].estado == 0) {
        memoria_libre_total += tabla_particiones[i].tamano;
    }
}

if (memoria_libre_total < 50) { /* Menos de 50 KB libres */
    printf("\nADVERTENCIA: Memoria disponible baja (%d KB)\n",
memoria_libre_total);
}
```

6. CAMBIAR CÁLCULO DE FRAGMENTACIÓN

✚ CAMBIO 14: Fragmentación en KB (No Porcentaje)

Modificar función (línea 270):

```
float calcular_fragmentacion(int tamano_particion, int tamano_proceso) {
    int fragmentacion;

    if (tamano_proceso == 0) {
        return 0.0;
    }

    fragmentacion = tamano_particion - tamano_proceso;
    return fragmentacion; /* ← Retornar KB directamente */
}
```

Modificar línea 326:

```
printf("    Fragmentacion interna: %.0f KB\n", fragmentacion); /* Sin % */
```

✦ CAMBIO 15: Mostrar Fragmentación Total

Agregar función:

```
void mostrar_fragmentacion_total(void) {
    int i;
    int frag_total_kb = 0;
    Proceso *proc;

    for (i = 0; i < num_particiones; i++) {
        if (tabla_particiones[i].estado == 1) {
            proc = buscar_proceso(tabla_particiones[i].id_proceso);
            if (proc != NULL) {
                frag_total_kb += (tabla_particiones[i].tamano - proc-
> tamano_requerido);
            }
        }
    }

    printf("\nFragmentacion interna total: %d KB desperdiciados\n",
frag_total_kb);
}
```

7. AGREGAR PRIORIDADES A PROCESOS

✦ CAMBIO 16: Procesos con Prioridad

Modificar estructura Proceso (línea 15):

```
typedef struct Proceso {
    int id;
    int tamano_requerido;
    int direccion_asignada;
    int prioridad; /* ← NUEVO: 1=Alta, 2=Media, 3=Baja */
    struct Proceso *siguiente;
} Proceso;
```

Modificar agregar_proceso () (línea 171):

```

void agregar_proceso(int id, int tamano, int direccion, int prioridad) {
    /* ... código anterior ... */

    nuevo->id = id;
    nuevo->tamano_requerido = tamano;
    nuevo->direccion_asignada = direccion;
    nuevo->prioridad = prioridad; /* ← NUEVO */
    nuevo->siguiente = NULL;

    /* ... resto del código ... */
}

```

Modificar crear_proceso() para generar prioridad (línea 290):

```

int prioridad = (rand() % 3) + 1; /* 1, 2 o 3 */
printf("Prioridad asignada: %d\n", prioridad);

```

Modificar llamada a agregar_proceso() (línea 319):

```

agregar_proceso(contador_procesos, tamano_proceso, dir_inicio, prioridad);

```

Modificar mostrar_tabla_procesos() (línea 388):

```

printf("%-10s %-15s %-20s %-12s\n", "ID", "Tamaño (KB)", "Direccion",
"Prioridad");
/* ... */
printf("%-10d %-15d %-20d %-12d\n",
    actual->id,
    actual->tamano_requerido,
    actual->direccion_asignada,
    actual->prioridad);

```

8. MODIFICAR MENÚ

✦ CAMBIO 17: Cambiar Orden del Menú

Modificar líneas 61-68:

```

printf(" 1. Ver Tabla de Procesos          \n"); /* Antes era 3 */
printf(" 2. Ver Tabla de Particiones       \n"); /* Antes era 4 */
printf(" 3. Crear Proceso                    \n"); /* Antes era 1 */
printf(" 4. Cerrar Proceso                   \n"); /* Antes era 2 */
printf(" 5. Ver Memoria Asignada              \n");
printf(" 6. Ver Procesos Terminados         \n");
printf(" 0. Salir                             \n");

```

Modificar switch (líneas 72-90) en consecuencia.

✦ CAMBIO 18: Agregar Opción "Limpiar Pantalla"

Agregar al menú:

```
printf(" 9. Limpiar Pantalla \n");
```

Agregar al switch:

```
case 9:
    system("clear"); /* En Linux */
    /* system("cls"); En Windows */
    break;
```

✦ CAMBIO 19: Confirmar antes de Salir

Modificar case 0 (línea 91):

```
case 0:
    printf("\n¿Esta seguro que desea salir? (S/N): ");
    char respuesta;
    scanf(" %c", &respuesta); /* Espacio antes de %c importante */

    if (respuesta == 'S' || respuesta == 's') {
        printf("\nSaliendo del programa...\n");
    } else {
        opcion = 1; /* No salir, continuar */
    }
    break;
```

9. CAMBIAR FORMATO DE SALIDA

✦ CAMBIO 20: Usar Marcos/Bordes Diferentes

Modificar línea 55:

```
/* Estilo simple */
printf("SIMULADOR DE GESTION DE MEMORIA\n");
printf("=====\n");
```

O estilo con asteriscos:

```
printf("*****\n");
printf("* SIMULADOR DE GESTION MEMORIA *\n");
printf("*****\n");
```

✦ CAMBIO 21: Agregar Timestamp a Procesos

Agregar al inicio:

```
#include <time.h> /* Ya está incluido */
```

Modificar estructura Proceso:

```
typedef struct Proceso {
    int id;
    int tamano_requerido;
```



```

    int direccion_asignada;
    time_t tiempo_creacion; /* ← NUEVO */
    struct Proceso *siguiente;
} Proceso;

```

Modificar agregar_proceso():

```
nuevo->tiempo_creacion = time(NULL); /* Guardar hora actual */
```

Modificar mostrar_tabla_procesos():

```

char buffer[30];
struct tm *info_tiempo;

info_tiempo = localtime(&actual->tiempo_creacion);
strftime(buffer, 30, "%H:%M:%S", info_tiempo);

printf("%-10d %-15d %-20d %-15s\n",
        actual->id,
        actual->tamano_requerido,
        actual->direccion_asignada,
        buffer); /* Muestra hora: 14:35:22 */

```

10. AGREGAR LÍMITES Y CONTROLES

✦ CAMBIO 22: Tamaño Máximo de Memoria

Modificar inicializar_memoria() (línea 114):

```

do {
    printf("Ingrese el tamaño total de memoria (KB, max 10000): ");
    scanf("%d", &tamano_total_memoria);
    if (tamano_total_memoria <= 0) {
        printf("Error: El tamaño debe ser mayor a 0.\n");
    } else if (tamano_total_memoria > 10000) {
        printf("Error: El tamaño maximo es 10000 KB.\n");
        tamano_total_memoria = 0; /* Forzar repetición */
    }
} while (tamano_total_memoria <= 0);

```

✦ CAMBIO 23: Contador de Operaciones

Agregar al inicio:

```

int contador_creaciones = 0;
int contador_cierres = 0;

```

Modificar crear_proceso() al final:

```

contador_creaciones++;
printf("      (Total procesos creados: %d)\n", contador_creaciones);

```

Modificar cerrar_proceso() al final:

```
contador_cierres++;  
printf("      (Total procesos cerrados: %d)\n", contador_cierres);
```

✦ CAMBIO 24: Modo Debug/Verbose

Agregar al inicio:

```
int modo_debug = 0; /* 0=Normal, 1=Debug */
```

Agregar al menú:

```
printf("  D. Activar/Desactivar modo Debug          \n");
```

Agregar al switch:

```
case 'D':  
case 'd':  
    modo_debug = !modo_debug; /* Toggle */  
    printf("\nModo debug: %s\n", modo_debug ? "ACTIVADO" : "DESACTIVADO");  
    break;
```

Agregar mensajes de debug en crear_proceso():

```
if (modo_debug) {  
    printf("DEBUG: Buscando particion para %d KB\n", tamano_proceso);  
    printf("DEBUG: Revisando %d particiones\n", num_particiones);  
}
```

11. CAMBIOS RÁPIDOS DE CONFIGURACIÓN

✦ CAMBIO 25: Configuración Automática

Agregar función:

```
void configuracion_rapida(void) {  
    printf("\n=== CONFIGURACION RAPIDA ===\n");  
    printf("1. Pequena (500 KB, 5 particiones)\n");  
    printf("2. Mediana (1000 KB, 10 particiones)\n");  
    printf("3. Grande (2000 KB, 20 particiones)\n");  
    printf("4. Personalizada\n");  
    printf("Opcion: ");  
  
    int opcion;  
    scanf("%d", &opcion);  
  
    switch(opcion) {  
        case 1:  
            tamano_total_memoria = 500;  
            tamano_particion = 100;  
            break;  
        case 2:  
            tamano_total_memoria = 1000;  
            tamano_particion = 100;
```

```

        break;
    case 3:
        tamano_total_memoria = 2000;
        tamano_particion = 100;
        break;
    case 4:
        /* Código normal de inicializar_memoria() */
        break;
}

num_particiones = tamano_total_memoria / tamano_particion;
/* ... resto de inicialización ... */
}

```

12. TABLA RESUMEN DE CAMBIOS

#	Cambio	Dificultad	Líneas a Modificar
1	First Fit → Best Fit	Media	296-303
2	First Fit → Worst Fit	Media	296-303
3	Selector de Algoritmo	Alta	Multiple
4	Tamaño Fijo	Fácil	290
5	Rango Personalizado	Fácil	290
6	Tamaños Predefinidos	Media	290 + nuevo array
7	Mostrar ID Real	Fácil	463
8	Colores	Media	460-463 + defines
9	Visual por Particiones	Alta	Reescribir función
10	Estadísticas	Alta	Nueva función
11	Frag. Promedio	Media	Nueva función
12	Límite de Procesos	Media	Al inicio de crear_proceso
13	Memoria Mínima	Media	Después de línea 290
14	Frag. en KB	Fácil	270-278
15	Frag. Total	Media	Nueva función
16	Prioridades	Alta	Modificar estructura
17	Orden del Menú	Fácil	61-68 + switch
18	Limpiar Pantalla	Fácil	Nuevo case
19	Confirmar Salir	Fácil	case 0
20	Bordes Diferentes	Fácil	55-56
21	Timestamp	Alta	Modificar estructura
22	Límite Máximo	Fácil	114-118
23	Contador Operaciones	Fácil	Variables globales
24	Modo Debug	Media	Variable global + prints
25	Config. Rápida	Alta	Nueva función

13. ESTRATEGIA PARA LA EVALUACIÓN

☒ ANTES DE LA EVALUACIÓN:

1. **Practica estos cambios** en tu computadora

2. **Memoriza las líneas clave:** 290 (tamaño), 296-303 (algoritmo), 463 (visualización)
3. **Ten el código impreso** con números de línea
4. **Marca con colores** las secciones importantes

☒ **DURANTE LA EVALUACIÓN:**

1. **Escucha bien** qué pide el ingeniero
2. **Identifica el tipo** de cambio (¿algoritmo? ¿visualización? ¿validación?)
3. **Ve directo a la línea** correspondiente
4. **Compila después** de cada cambio para verificar

☒ **FRASES CLAVE QUE PUEDE DECIR EL INGENIERO:**

Frase del Ingeniero	Lo que debes hacer
"Cambia a Best Fit"	CAMBIO #1
"Que no sea aleatorio"	CAMBIO #4
"Muestra el ID real"	CAMBIO #7
"Agrega estadísticas"	CAMBIO #10
"Limita los procesos"	CAMBIO #12
"Cambia la visualización"	CAMBIO #7, 8 o 9
"Agrega validación de..."	CAMBIO #12, 13, 22
"Modifica el menú"	CAMBIO #17, 18

14. CÓDIGO DE EMERGENCIA (Copy-Paste Rápido)

Para Best Fit:

```
particion_asignada = -1;
int menor_desperdicio = tamano_total_memoria + 1;
int desperdicio;
for (i = 0; i < num_particiones; i++) {
    if (tabla_particiones[i].estado == 0 && tabla_particiones[i].tamano >=
tamano_proceso) {
        desperdicio = tabla_particiones[i].tamano - tamano_proceso;
        if (desperdicio < menor_desperdicio) {
            menor_desperdicio = desperdicio;
            particion_asignada = i;
        }
    }
}
```

Para Mostrar ID Real:

```
printf("[%d] ", memoria[i]);
```

Para Tamaño Fijo 50 KB:

```
tamano_proceso = 50;
```

Para Estadísticas:

```
/* Contar procesos activos */
```

```
int num_activos = 0;
Proceso *p = lista_procesos;
while (p != NULL) { num_activos++; p = p->siguiente; }
printf("Procesos activos: %d\n", num_activos);
```

15. TIPS FINALES

1. **Compila frecuentemente:** Después de CADA cambio, compila para verificar
2. **Guarda versiones:** Antes de cambiar, guarda una copia del original
3. **Comenta tus cambios:** Agrega /* MODIFICADO */ para ubicarte rápido
4. **Prueba el programa:** Ejecuta y verifica que funcione correctamente
5. **Conoce tu código:** Entiende CADA línea, no solo copies